

CPE 324-03: Advanced Digital Logic Design Laboratory

Final Project Alarm Clock Module

Submitted by: Hannah Kelley

Date of Experiment: April 7, 9, 14, 16, and 21, 2026

Report Deadline: April 23, 2026

Demonstration Deadline: April 21, 2026

Lab Partner: Margaret Roberts

1 Introduction

This project focuses on the design and implementation of a digital alarm clock module using the DE2-115 educational trainer. The objective is to build a fully functional clock system that displays the current time in hours, minutes, and seconds while maintaining high accuracy. In addition to basic timekeeping, the system includes user-friendly controls for setting both the current time and an alarm time independently. When the set alarm time is reached, the module generates an audible or audio-visual alert that can be manually reset by the user. This project builds upon concepts from previous labs and integrates provided IP code samples and previous lab solutions to create the final design.

2 Experiment Description

The project requirements for the alarm clock project are as follows:

- The Alarm Clock Module should continuously visually display the time of day, showing the hours, minutes and seconds. Time can be in either 24 hour format or 12 hour format. In the case of 12 hour format there must be a AM and PM indicator.
- Time should be accurate to within two minutes per year. Students should verify this accuracy by referring to the specifications of the DE-2 115 and its data sheets.
- There is to be an ergonomically designed mechanism for the user to manually set the time.
- There is to be a similar method to for the user to set an alarm time which the clock module will remember and use to generate an alarm signal when the current time of day is equal to the alarm time.
- The alarm time should be able to be set by the user without affecting the clock's main time of day operation (it is okay though if the display switches over to the alarm time during the alarm set operation)
- The alarm should be an audible signal or combined audio-visual signal that is loud enough to wake the user.
- The alarm signal should be able to be reset by the user by pressing a button.
- The maximum time for this alarm signal should be on if not reset by the user is 5 minutes.
- The alarm clock module should be prototyped on the DE2-115 Educational Trainer.

The digital alarm clock was implemented using the 50 MHz system clock available on the DE2-115 FPGA. The design is divided into several functional components, including timekeeping, display features, counting logic, clock and alarm setting mechanisms, and alarm control.

The timekeeping mechanism is driven by the 50 MHz input clock, `CLOCK_50`, which is built into the board. A cycle counter is used to divide this high-frequency clock down to a 1 Hz signal by incrementing a register (cycles) and updating the time whenever the count reaches

50,000,000. At this point, the system increments the time by one second. The current time is stored in three registers representing seconds, minutes, and hours.

Additional logic was implemented to ensure proper time progression and wraparound behavior. A 24-hour format is used, where seconds and minutes increment from 0 to 59 before resetting to 0, and hours increment from 0 to 23 before wrapping back to 0. This cascading structure ensures accurate and continuous timekeeping.

To display the time, the current values are continuously converted from binary to BCD format using the `bintobcd` function developed in previous labs. These BCD values are then translated into seven-segment display outputs using the `bintohehex` task, also reused from earlier work. The resulting signals drive the HEX displays on the FPGA. Under normal operation, the display shows the current time; however, during alarm set mode, the display switches to show the configured alarm time instead.

The clock set feature is controlled using switch 0. When activated, the system enters a time-setting mode in which normal time progression is paused. Keys 1 through 3 are used to increment the seconds, minutes, and hours, respectively, while key 0 resets the time to zero. Each increment follows the same wraparound logic used during normal operation. All user inputs are debounced using a provided IP module to ensure stable and reliable operation.

The alarm set feature is controlled using switch 1. When this mode is active, the user can independently configure the alarm time without affecting the main clock operation. Similar to the clock set mode, keys 1 through 3 increment the alarm seconds, minutes, and hours, while key 0 resets the alarm time. The alarm values are stored in separate registers, ensuring independence between the current time and the alarm configuration.

The alarm is triggered when the current time matches the stored alarm time. By default, the alarm time is initialized to 12 hours, 0 minutes, and 0 seconds. When activated, the output signal `d_bit` toggles at a rate derived from `CLOCK_50`, producing an audible signal through the connected speaker.

Finally, the alarm reset functionality is implemented in two ways. The user can manually disable the alarm by pressing key 0, which immediately clears the alarm signal and stops the sound output. Alternatively, an automatic timeout mechanism disables the alarm after five minutes if it is not manually reset.

Figure 1 gives the pin configuration for this project below.

	Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard
in	CLOCK_50	Input	PIN_Y2	2	B2_N0	PIN_Y2	2.5 V
out	HEX2[6]	Output	PIN_W28	5	B5_N1	PIN_W28	2.5 V
out	HEX2[5]	Output	PIN_W27	5	B5_N1	PIN_W27	2.5 V
out	HEX2[4]	Output	PIN_Y26	5	B5_N1	PIN_Y26	2.5 V
out	HEX2[3]	Output	PIN_W26	5	B5_N1	PIN_W26	2.5 V
out	HEX2[2]	Output	PIN_Y25	5	B5_N1	PIN_Y25	2.5 V
out	HEX2[1]	Output	PIN_AA26	5	B5_N1	PIN_AA26	2.5 V
out	HEX2[0]	Output	PIN_AA25	5	B5_N1	PIN_AA25	2.5 V
out	HEX3[6]	Output	PIN_Y19	4	B4_N0	PIN_Y19	2.5 V
out	HEX3[5]	Output	PIN_AF23	4	B4_N0	PIN_AF23	2.5 V
out	HEX3[4]	Output	PIN_AD24	4	B4_N0	PIN_AD24	2.5 V
out	HEX3[3]	Output	PIN_AA21	4	B4_N0	PIN_AA21	2.5 V
out	HEX3[2]	Output	PIN_AB20	4	B4_N0	PIN_AB20	2.5 V
out	HEX3[1]	Output	PIN_U21	5	B5_N0	PIN_U21	2.5 V
out	HEX3[0]	Output	PIN_V21	5	B5_N1	PIN_V21	2.5 V
out	HEX4[6]	Output	PIN_AE18	4	B4_N2	PIN_AE18	2.5 V
out	HEX4[5]	Output	PIN_AF19	4	B4_N1	PIN_AF19	2.5 V
out	HEX4[4]	Output	PIN_AE19	4	B4_N1	PIN_AE19	2.5 V
out	HEX4[3]	Output	PIN_AH21	4	B4_N2	PIN_AH21	2.5 V
out	HEX4[2]	Output	PIN_AG21	4	B4_N2	PIN_AG21	2.5 V
out	HEX4[1]	Output	PIN_AA19	4	B4_N0	PIN_AA19	2.5 V
out	HEX4[0]	Output	PIN_AB19	4	B4_N0	PIN_AB19	2.5 V
out	HEX5[6]	Output	PIN_AH18	4	B4_N2	PIN_AH18	2.5 V
out	HEX5[5]	Output	PIN_AF18	4	B4_N1	PIN_AF18	2.5 V
out	HEX5[4]	Output	PIN_AG19	4	B4_N2	PIN_AG19	2.5 V
out	HEX5[3]	Output	PIN_AH19	4	B4_N2	PIN_AH19	2.5 V
out	HEX5[2]	Output	PIN_AB18	4	B4_N0	PIN_AB18	2.5 V
out	HEX5[1]	Output	PIN_AC18	4	B4_N1	PIN_AC18	2.5 V
out	HEX5[0]	Output	PIN_AD18	4	B4_N1	PIN_AD18	2.5 V
out	HEX6[6]	Output	PIN_AC17	4	B4_N2	PIN_AC17	2.5 V
out	HEX6[5]	Output	PIN_AA15	4	B4_N2	PIN_AA15	2.5 V
out	HEX6[4]	Output	PIN_AB15	4	B4_N2	PIN_AB15	2.5 V
out	HEX6[3]	Output	PIN_AB17	4	B4_N1	PIN_AB17	2.5 V
out	HEX6[2]	Output	PIN_AA16	4	B4_N2	PIN_AA16	2.5 V
out	HEX6[1]	Output	PIN_AB16	4	B4_N2	PIN_AB16	2.5 V
out	HEX6[0]	Output	PIN_AA17	4	B4_N1	PIN_AA17	2.5 V
out	HEX7[6]	Output	PIN_AA14	3	B3_N0	PIN_AA14	2.5 V
out	HEX7[5]	Output	PIN_AG18	4	B4_N2	PIN_AG18	2.5 V
out	HEX7[4]	Output	PIN_AF17	4	B4_N2	PIN_AF17	2.5 V
out	HEX7[3]	Output	PIN_AH17	4	B4_N2	PIN_AH17	2.5 V

Figure 1: Pin Configuration

Table 2 gives the keys and switches used in this project with their uses in each operating mode.\

Switch/Key	Mode	Functionality
Key 0	Normal Operation	Turn off the alarm signal and speaker
Switch 1	Normal Operation/Time Set Mode	Toggle time set mode
Switch 2	Normal Operation/Alarm Set Mode	Toggle alarm set mode
Key 0	Time Set Mode	Reset time to 00:00:00
Key 1	Time Set Mode	Increment seconds
Key 2	Time Set Mode	Increment minutes
Key 3	Time Set Mode	Increment hours
Key 0	Alarm Set Mode	Reset time to 00:00:00
Key 1	Alarm Set Mode	Increment seconds
Key 2	Alarm Set Mode	Increment minutes
Key 3	Alarm Set Mode	Increment hours

Table 2: Input Devices and their Functionality

3 Results

Figure 2 shows the compilation results and the FPGA resource allocation for the final design. Figure 3 shows the digital alarm clock and speaker wiring configuration. Figure 4 shows the alarm clock in normal operation.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Tue Apr 21 09:38:45 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	alarm_clock
Top-level Entity Name	alarm_clock
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	3,158 / 114,480 (3 %)
Total registers	159
Total pins	50 / 529 (9 %)
Total virtual pins	0
Total memory bits	0 / 3,981,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	0 / 4 (0 %)

Figure 2: Compilation Results and FPGA Resource Allocation

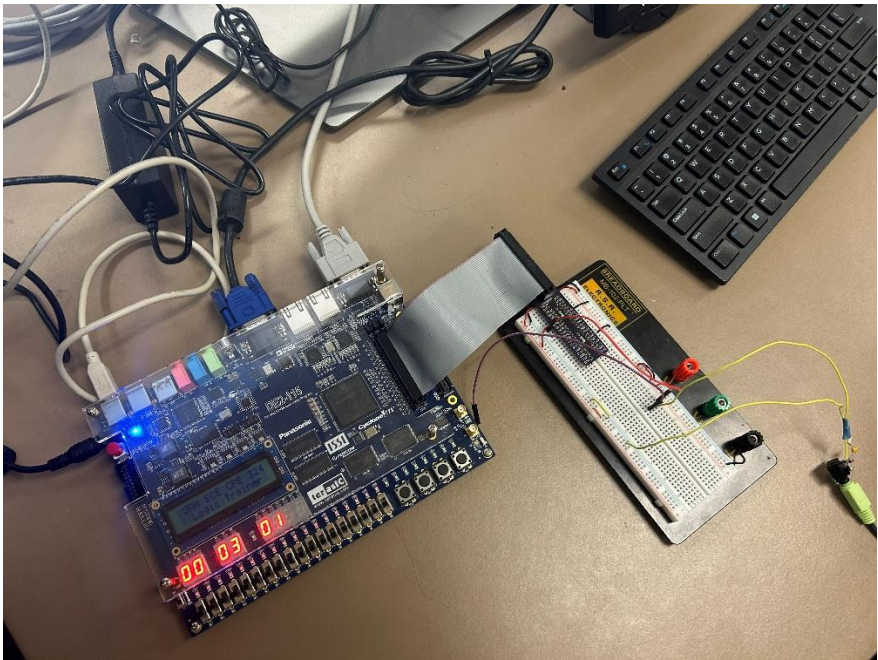


Figure 3: Wiring Configuration

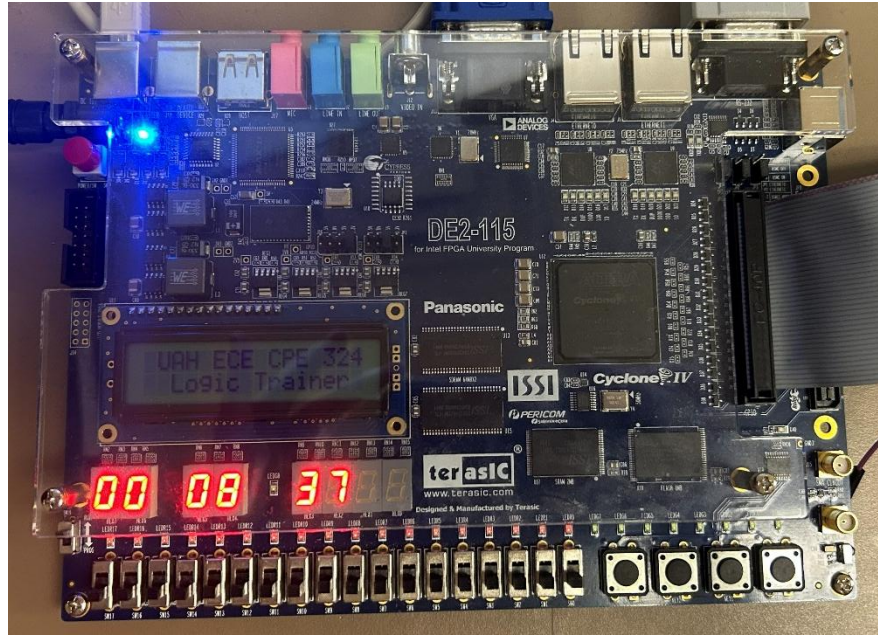


Figure 4: Alarm Clock in Normal Operation

The project was demonstrated to the TA on April 21, 2026 with full functionality.

4 Conclusion

This lab successfully demonstrated the design and implementation of a digital alarm clock on the DE2-115 FPGA platform. The system integrated multiple functional blocks, including timekeeping, user input handling, display control, and alarm generation. The final design was able to maintain accurate time progression, support manual time and alarm configuration, and trigger an audible alert when the programmed alarm time was reached. Additionally, the wraparound logic and mode-based operation ensured reliable and continuous functionality across different operating states.

One of the main challenges encountered during implementation involved the alarm output system, specifically configuring and debugging the speaker interface. Significant time was spent troubleshooting the absence of an audible alarm signal, which ultimately was traced to the speaker not being physically connected to the board. This highlighted the importance of verifying both hardware setup and software functionality during system-level debugging. Once resolved, the alarm output operated as expected.

Overall, the project reinforced key concepts in digital system design, including clock division, synchronous state behavior, input debouncing, and FPGA-based hardware integration.

Appendix A – Complete alarm_clock.v File

```
module alarm_clock(
    input CLOCK_50,
    input [3:0] KEY,
    input [1:0] SW,
    output [6:0] HEX7, HEX6, HEX5, HEX4, HEX3, HEX2,
    output reg d_bit
);

    reg [41:0] cycles;
    reg [5:0] sec, min, hour;
    reg [5:0] a_sec, a_min, a_hour;
    reg [7:0] sec_bcd, min_bcd, hour_bcd;
    reg alarm_on, alarm_past;
    reg active, count;
    reg sw0_prev, sw1_prev;
    reg k3_prev, k2_prev, k1_prev, k0_prev;

    wire not_key0, not_key1, not_key2, not_key3;
    wire k3, k2, k1, k0;
    wire sw0, sw1;

    assign not_key0 = ~KEY[0];
    assign not_key1 = ~KEY[1];
    assign not_key2 = ~KEY[2];
    assign not_key3 = ~KEY[3];

    debounce C0 (.clk(CLOCK_50), .d_in(not_key3), .d_out(k3));
    debounce C1 (.clk(CLOCK_50), .d_in(not_key2), .d_out(k2));
    debounce C2 (.clk(CLOCK_50), .d_in(not_key1), .d_out(k1));
    debounce C4 (.clk(CLOCK_50), .d_in(not_key0), .d_out(k0));
    debounce C5 (.clk(CLOCK_50), .d_in(SW[0]), .d_out(sw0));
    debounce C6 (.clk(CLOCK_50), .d_in(SW[1]), .d_out(sw1));

    initial begin
        active = 1;
        alarm_on = 0;
        cycles = 0;
        hour = 0;
        min = 0;
        sec = 0;
        d_bit = 0;
    end

    always @(posedge CLOCK_50) begin
        sw0_prev <= sw0;
```

```

sw1_prev <= sw1;
k3_prev <= k3;
k2_prev <= k2;
k1_prev <= k1;

if ((hour == a_hour) && (min == a_min) && (sec == a_sec))
    alarm_on <= 1;

if (((hour == a_hour) && (min == (a_min + 5)) && (sec == a_sec)) || (k0))
    alarm_on <= 0;

if (alarm_on && (cycles % 50000 == 0)) begin
    d_bit <= ~d_bit;
end

if (~sw0_prev && sw0) begin
    active <= 0;
    hour <= 0;
    min <= 0;
    sec <= 0;
end
else if (sw0) begin
    active <= 0;

    if (k3 && !k3_prev) begin
        if (hour < 23) hour <= hour + 1;
        else hour <= 0;
    end

    if (k2 && !k2_prev) begin
        if (min < 59) min <= min + 1;
        else min <= 0;
    end

    if (k1 && !k1_prev) begin
        if (sec < 59) sec <= sec + 1;
        else sec <= 0;
    end

    if (k0) begin
        hour <= 0;
        min <= 0;
        sec <= 0;
    end
end
else begin

```



```

    active <= 1;
end

if (sw1) begin
    active <= 1;

    if (k3 && !k3_prev) begin
        if (a_hour < 23) a_hour <= a_hour + 1;
        else a_hour <= 0;
    end

    if (k2 && !k2_prev) begin
        if (a_min < 59) a_min <= a_min + 1;
        else a_min <= 0;
    end

    if (k1 && !k1_prev) begin
        if (a_sec < 59) a_sec <= a_sec + 1;
        else a_sec <= 0;
    end

    if (k0) begin
        a_hour <= 0;
        a_min <= 0;
        a_sec <= 0;
    end
end

if (active) begin
    cycles <= cycles + 1;

    if (cycles % 50000000 == 0) begin
        if (sec == 59) begin
            sec <= 0;

            if (min == 59) begin
                min <= 0;

                if (hour == 23) begin
                    hour <= 0;
                    min <= 0;
                    sec <= 0;
                end
            else begin
                hour <= hour + 1;
            end
        end
    end
end

```

```

        end
        else begin
            min <= min + 1;
        end
    end
    else begin
        sec <= sec + 1;
    end
end
end
end

always @(*) begin
    if (~sw1) begin
        sec_bcd <= bintobcd(sec);
        min_bcd <= bintobcd(min);
        hour_bcd <= bintobcd(hour);
    end
    else begin
        sec_bcd <= bintobcd(a_sec);
        min_bcd <= bintobcd(a_min);
        hour_bcd <= bintobcd(a_hour);
    end

    bintohehex(sec_bcd[3:0], HEX2);
    bintohehex(sec_bcd[7:4], HEX3);

    bintohehex(min_bcd[3:0], HEX4);
    bintohehex(min_bcd[7:4], HEX5);

    bintohehex(hour_bcd[3:0], HEX6);
    bintohehex(hour_bcd[7:4], HEX7);
end

task bintohehex(input [3:0] bin_num, output reg [6:0] hex_num);
    case (bin_num)
        0 : hex_num = 7'b1000000;
        1 : hex_num = 7'b1111001;
        2 : hex_num = 7'b0100100;
        3 : hex_num = 7'b0110000;
        4 : hex_num = 7'b0011001;
        5 : hex_num = 7'b0010010;
        6 : hex_num = 7'b0000010;
        7 : hex_num = 7'b1111000;
        8 : hex_num = 7'b0000000;
        9 : hex_num = 7'b0011000;
    endcase
endtask

```

```

    10 : hex_num = 7'b0001000;
    11 : hex_num = 7'b00000011;
    12 : hex_num = 7'b1000110;
    13 : hex_num = 7'b0100001;
    14 : hex_num = 7'b0000110;
    15 : hex_num = 7'b0001110;
    default : hex_num = 7'bx;

```

```

endcase
endtask

```

```

function [7:0] bintobcd;
    input [5:0] bin_num;
    reg [7:0] bcd_num = 0;
    reg [5:0] residual;
    reg [1:0] i;

    for (i = 0; i <= 1; i = i + 1) begin
        residual = bin_num / 10;
        bcd_num = bcd_num + ((bin_num - residual * 10) << (4 * i));
        bin_num = residual;
    end

```

```

    bintobcd = bcd_num;
endfunction

```

```

endmodule

```

```

module debounce (
    input clk,
    input d_in,
    output reg d_out
);

```

```

    always @(posedge clk) begin : MAIN_LOOP
        reg [7:0] hld_sig = 8'd0;
        reg [4:0] index = 0;
        reg [14:0] count = 15'd0;
        reg d_st = 0;

        if (count == 15'd32767) begin
            count = 0;

            hld_sig[index] = d_in;

            if (index == 7)

```

```
        index = 0;
    else
        index = index + 1;

    if (!d_st) begin
        d_st = hld_sig[0] & hld_sig[1] & hld_sig[2] & hld_sig[3] &
            hld_sig[4] & hld_sig[5] & hld_sig[6] & hld_sig[7];
        d_out = d_st;
    end
    else begin
        d_st = hld_sig[0] | hld_sig[1] | hld_sig[2] | hld_sig[3] |
            hld_sig[4] | hld_sig[5] | hld_sig[6] | hld_sig[7];
        d_out = d_st;
    end
end
else begin
    count = count + 1;
end
end
endmodule
```